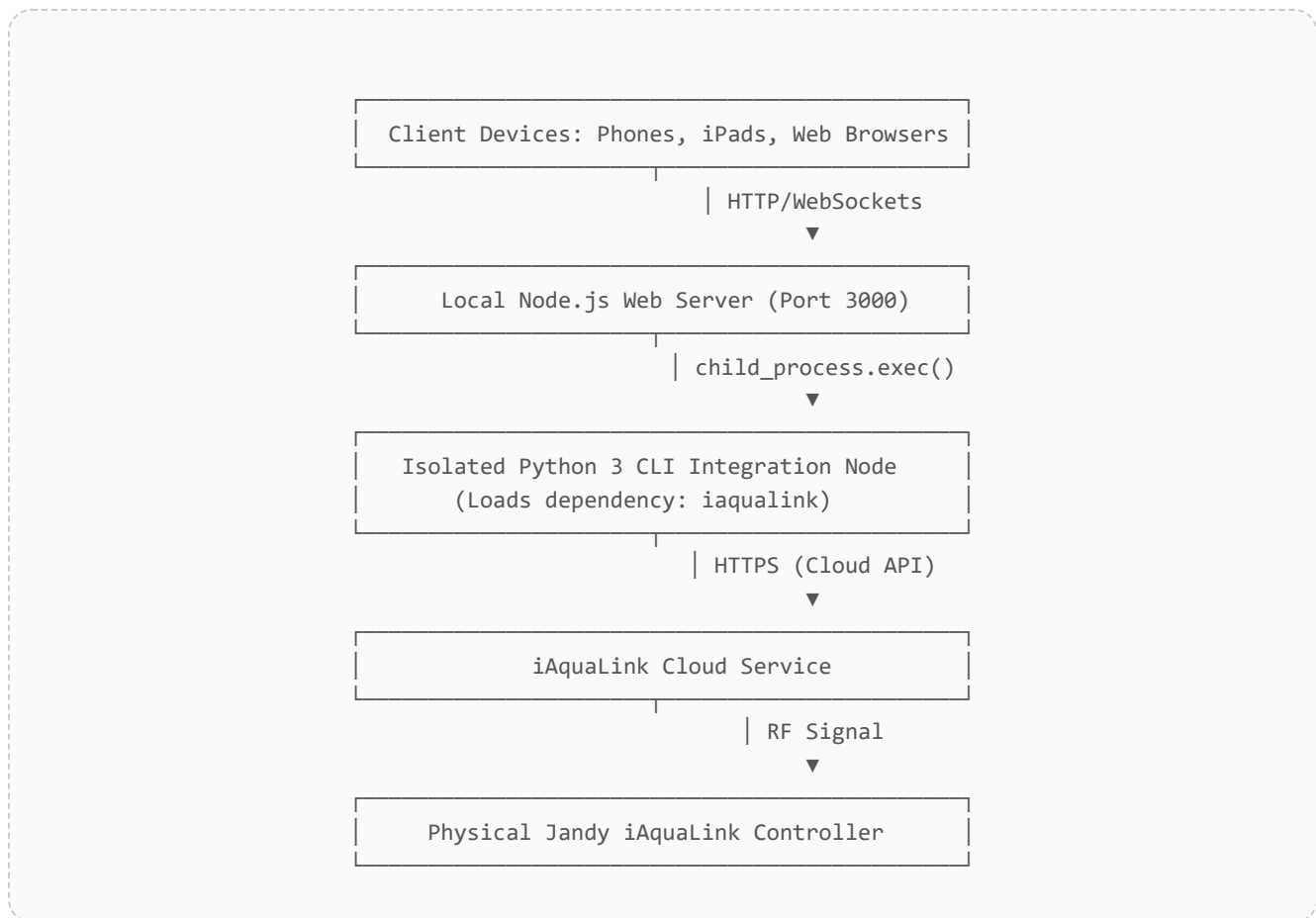


TECHNICAL SPECIFICATION

PoolPilot Architecture & Technical Details

PoolPilot is a self-hosted, lightweight home automation server that bridges Jandy iAquaLink pool controllers with modern and legacy web dashboards. It runs locally on your home network and requires no cloud subscriptions, database setup, or external dependencies.



1. The Core Integration: The `iaqualink` Python Library

At the center of hardware communication is the open-source `iaqualink` Python library. This library maps pool hardware relays to clean object models in code:

- **Telemetry Retrieval (`get_pool_temp.py`)**: Polls the cloud state to fetch live readings including air temperature, water temperature, target setpoints, pump speeds, heater status, and light colors/shows.

- **State Controls (`toggle_device.py`)**: Connects to the API to command state changes such as toggling relay switches on or off, or selecting active Jandy LED WaterColors shows.

ZERO-CONFIG LOCAL PYTHON DEPENDENCY ISOLATION

To avoid global package pollution, PoolPilot installs python libraries locally. On `npm install`, the server runs a custom postinstall script (`install-pip.js`) that downloads `get-pip.py` and builds an isolated dependency folder (`.python_lib/`). When running scripts, the Node server sets `PYTHONPATH=.python_lib`, keeping the host system completely clean.

2. Node.js & Express Server

The host server script is written in TypeScript and runs via Node.js/Express. It performs three critical roles:

1. **LAN Host Broadcaster**: Binds to the network address `0.0.0.0` on port `3000`. This enables any device on your local Wi-Fi to load the UI by navigating to the host server's local IP address.
2. **API Middleware Router**:
 - `GET /api/config`: Reads local configuration, resolves the host server's network IPv4 address using Node's `os` module, and outputs it to client screens.
 - `POST /api/config`: Receives credentials. It verifies connection status using the Python connector before writing credentials to the local `config.json` file.
 - `GET /api/pool-temp`: Spawns the python telemetry subprocess, captures stdout, and sends the parsed JSON response back to the browser.
 - `POST /api/toggle`: Executes python relays toggle commands in real-time.
3. **Browser Auto-Launcher**: Upon running `npm run start`, the server auto-detects the operating system (Windows, MacOS, or Linux) and launches the default web browser directed to the application home screen.

3. Dual-Frontend Dashboards

PoolPilot includes two frontend interfaces to accommodate different eras of display hardware:

A. Modern Dashboard (React, TypeScript, TailwindCSS v4)

Optimized for modern smartphones, laptops, and newer wall-mounted screens. Features visual enhancements such as dark mode glassmorphism, gradient accents, pulsing alert statuses, and interactive loaders. Codebase is fully type-safe.

B. Legacy Dashboard (ES5 HTML5 / Vanilla CSS)

Written specifically to breathing new life into old, repurposed tablets (such as a 1st/2nd Gen iPad Mini locked to iOS 9/10). Code is written in strict ES5 standards (utilizing `var` , standard functions, and standard `XMLHttpRequest` instead of modern arrow functions or the `fetch` API) and employs a robust CSS Flexbox grid layout suitable for older web rendering engines.

4. Screen Personalization via Local Storage

PoolPilot divides configuration parameters into two layers:

- **Server-Level Settings (`config.json`)**: Holds credentials and globally active pool devices. Disabled items are omitted system-wide.
- **Device-Level Visibility (Browser `localStorage`)**: Stores layout preferences (card show/hide toggles) on each specific device. This allows you to personalize what is shown on a kitchen iPad (e.g. only pool temp and light buttons) versus a master bedroom display.